ETIB

Interpreter Self-Training Platform

Ecole de Traducteurs et d'Interpretes de Beyrouth | USJ Beirut

Technical Documentation

Developer Reference — Architecture, Setup, API

Working languages: Arabic | French | English

Final Year Project - ESIB, Universite Saint-Joseph

Supervised by Prof. Lina Sader Feghali and Prof. Wadad Wazen Gergy

Generated June 24, 2026

Project Overview

The ETIB Interpreter Self-Training Platform is an AI-powered web application that lets trainee conference interpreters practice independently. Students generate realistic training speeches, listen to them via text-to-speech, record their own interpretation, and receive instant automated feedback — all in one browser session.

Module	What it produces
A Speech Generation	Configurable training speech (topic, domain, length, difficulty, hesitations, number density, discourse structure) grounded in real UN documents when available
B Audio & Materials	Edge-TTS audio with accent/speed control + key terms, thematic summary, MCQ, comprehension questions, trilingual glossary (AR/FR/EN), downloadable DOCX
C Transcription	Groq Whisper-large-v3 ASR (falls back to local faster-whisper) + Arabic tashkeel of what the student actually said
D Evaluation	Hesitation count, omission detection, number errors, terminology coverage, pronunciation alignment, LLM feedback paragraph, adaptive difficulty recommendations
E Progress	Session history, score trend (last 10 sessions), focus areas, strengths, specific recurring errors

Architecture

```
Browser (React + Vite)
  | fetch() with X-Groq-Api-Key header
  v
Flask backend  (Python 3.11)
  +-- before_request  ->  flask.g.groq_api_key
  +-- /api/module-a   ->  LLM speech generation   (llm_service.py -> Groq)
  +-- /api/module-b   ->  TTS (edge-tts) + materials (Groq)
  +-- /api/module-c   ->  ASR transcription (Groq Whisper / faster-whisper)
  +-- /api/module-d   ->  Evaluation + feedback (Groq) + sessions (MongoDB)
  +-- /api/library    ->  UN Digital Library search + PDF download
  +-- /api/auth       ->  Login / signup / validate-groq-key
```

No server-side Groq key — each student supplies their own free key via the Settings modal. The key lives only in their browser (localStorage) and is sent per-request as the X-Groq-Api-Key HTTP header.

Tech Stack

Layer	Technology
Frontend	React 18, Vite, plain CSS (no framework)

Backend

Flask 3, Python 3.11

LLM

Groq API — llama-3.3-70b-versatile (speech gen, eval, feedback)

ASR

Groq hosted whisper-large-v3 -> local faster-whisper fallback

TTS

edge-tts (Microsoft Azure Neural voices, free)

Embeddings

paraphrase-multilingual-MiniLM-L12-v2 (sentence-transformers) for RAG

Database

MongoDB (local) with in-memory fallback for dev

UN Library

UN Digital Library MARCXML API + curl PDF download

Local Setup

Prerequisites

- Python 3.10+
- Node.js 18+
- ffmpeg (winget install ffmpeg / brew install ffmpeg / sudo apt install ffmpeg)
- MongoDB (optional — app runs without it, sessions stored in memory)

1 — Clone and configure

```
git clone https://github.com/chrisswhb/ETIB-Interpreter-Trainer.git
cd ETIB-Interpreter-Trainer
git checkout joe-main
```

Create backend/.env :

```
LLM_PROVIDER=groq
FLASK_SECRET_KEY=change-me-in-production
FLASK_DEBUG=true
UPLOAD_FOLDER=./uploads
AUDIO_OUTPUT_FOLDER=./audio_outputs
# GROQ_API_KEY intentionally omitted — students supply their own key
```

2 — Backend

```
cd backend
python -m venv venv
venv\Scripts\activate        # Windows
# source venv/bin/activate    # macOS / Linux
pip install -r requirements.txt
python app.py
# -> http://127.0.0.1:5000
```

3 — Frontend

```
cd Frontend
npm install
npm run dev
# -> http://localhost:5173
```

Per-Student API Key Model

The flow for every Groq-backed request:

```
1. Student saves their key in Settings modal
   -> localStorage: etib_groq_api_key = "gsk_..."

2. Every fetch() in api.js calls groqHeaders()
   -> adds X-Groq-Api-Key: gsk_...  to the HTTP request

3. Flask before_request hook in app.py reads the header
   -> flask.g.groq_api_key = key

4. get_groq_client() in backend/utils/groq_client.py
   -> reads flask.g.groq_api_key
   -> raises RuntimeError if None (no key = no generation)
   -> creates Groq(api_key=key) and caches it per key

5. Groq call is charged to the student's own account
```

> *To restore the shared server-key approach (one key for all students), switch to branch: api-key-commun*

Groq Free Tier Limits (as of 2026)

Model	Req / min	Tokens / min	Tokens / day
llama-3.3-70b-versatile	30	12 000	100 000
whisper-large-v3	20	—	2 000 audio-sec

A typical session (generate + transcribe + evaluate) uses approximately 3 000 - 5 000 tokens. The free tier supports roughly 20-30 full sessions per day per student key.

Project Structure

```
ETIB-Interpreter-Trainer/
|-- backend/
|   |-- app.py                  Flask app, blueprints, CORS, before_request
|   |-- config.py               All env vars and constants
|   |-- requirements.txt
|   |-- modules/
|   |   |-- module_a.py         Speech generation (LLM + RAG + UN grounding)
|   |   |-- module_b.py         TTS + pedagogical materials
|   |   |-- module_c.py         ASR transcription + tashkeel
|   |   |-- module_d.py         Evaluation, feedback, sessions, adaptive params
|   |   |-- module_library.py   UN Digital Library search + fetch
|   |   |-- alignment.py        WhisperX forced alignment + LLM analysis
|   |   `-- auth.py             Login, signup, validate-groq-key
|   |-- services/
|   |   `-- llm_service.py      LLM provider abstraction (Groq / Gemini / local)
|   `-- utils/
|       `-- groq_client.py      Per-request Groq client factory
|-- Frontend/
|   |-- src/
|   |   |-- App.jsx             All React components + UI strings (EN/AR/FR)
|   |   `-- api.js              All fetch helpers with groqHeaders()
|   |-- styles/
|   |   |-- main.css
|   |   `-- rtl.css
|   `-- index.html
`-- docs/
    |-- USER_GUIDE.md
    `-- ETIB_User_Guide.pdf
```

Branches

Branch	Purpose
joe-main	Main development branch — all features, per-student key
api-key-commun	Backup: shared server-key approach (one key for all students)
kevin-main	Kevin's contributions (merged into joe-main)

Module A — Speech Generation (Technical Notes)

RAG pipeline

Used when a source document or UN PDF is provided:

- Source document chunked: 1 800 characters per chunk, 250-character overlap
- Each chunk embedded with paraphrase-multilingual-MiniLM-L12-v2
- Cosine similarity computed between query and all chunks
- Top-4 chunks injected into the LLM prompt as context

UN grounding

- Searches UN Digital Library MARCXML API (tries 5 results, 3 languages: EN/FR/ES)
- Downloads PDF using curl with browser User-Agent headers (WAF bypass)
- 20-second per-PDF timeout, 40-word minimum to count as valid
- Falls back to ungrounded generation with factual accuracy rules if no document found

Arabic specifics

- All digit sequences converted to Eastern Arabic-Indic numerals (0-9 -> Eastern form)
- Prompt instructs LLM to use correct Arabic numeral forms

Key endpoint

```
POST /api/module-a/generate
POST /api/module-a/from-document
POST /api/module-a/retrieve-document-context
```

Module C — ASR Transcription (Technical Notes)

- Primary: Groq hosted whisper-large-v3 (~3 seconds per recording)
- Fallback: local faster-whisper (medium model, ~1-3 min on CPU)
- Disfluency priming prompts per language to preserve hesitation markers
- Arabic tashkeel added post-transcription: reflects what student actually said, including errors
- WhisperX forced alignment available if installed: word-level confidence scores

Key endpoint

```
POST /api/module-c/transcribe
POST /api/module-c/align
GET  /api/module-c/status
```

Module D — Evaluation (Technical Notes)

Error detection

- Hesitations: regex matching per language (euh, um, ah, yani, ...)
- Number errors: number token comparison between source and transcript
- Omissions: silence gaps > 500 ms in audio flagged as possible omissions
- Terminology: key terms from source checked for presence in transcript

Adaptive parameters

- Recomputed after each session from score history
- Tracks problems_to_work_on and top_errors across sessions
- Recommends difficulty / length / domain for next session

Session storage

- Stored in MongoDB collection: etib_interpreter_trainer.sessions
- Falls back to in-memory dict if MongoDB is unavailable
- Sessions keyed by user_id (or "anonymous" if not authenticated)

Key endpoints

```
POST /api/module-d/full-evaluation
POST /api/module-d/evaluate
POST /api/module-d/feedback
GET  /api/module-d/sessions
GET  /api/module-d/adaptive-params
```

Authentication Endpoints

Endpoint	Method	Description
POST /api/auth/signup	POST	Create a new account
POST /api/auth/login	POST	Log in with email + password
GET /api/auth/me	GET	Check current session
POST /api/auth/logout	POST	End the session
POST /api/auth/validate-groq-key	POST	Test a Groq API key (live call)

> *Session uses Flask server-side sessions (cookie). MongoDB stores user accounts when available; in-memory dict otherwise.*